M26CS4.301. Data and Applications

Team 54

Project Phase 3

Arush Sharma, 2025101011
Vinayak Sharma, 2025114002
Pranjal Baranwal, 2025114004
Sai Sreemayee Sistla, 2025114013

Assigned Phase 2 submission: Team 63 (EER model of Team 67's "Database for speedrun.com")
References to Team 67's document use its section and requirement IDs (for example §2.4, U1).
References to Team 63's document use its finding IDs (for example F1, §(e) 5.1).

1. Stage A: ER-to-Relational Mapping

1.1 Overview

We mapped the final version of Team 63's EER diagram using the textbook algorithm (Elmasri & Navathe, Ch. 9). This version includes Team 63's only structural fix, F1, which adds REPORTS_RUN between RUN and TICKET. The result has 35 relations. The relational model keeps every entity type, every relationship type and every stored attribute of the diagram. The only attributes not stored are the derived ones listed in §1.4.

1.2 Strong entity types (Step 1)

Each strong entity type became one relation, with its key as the primary key:

- GAME, SERIES, PLATFORM, REGION, USER, RUN, MODERATOR_ROLE, FORUM, NEWS_POST, STREAM, MEMBERSHIP, TICKET
- Second keys are declared UNIQUE:
- GAME.abbreviation
- USER.username and USER.email
- MODERATOR_ROLE.role_name
- FORUM.forum_name

1.3 Weak entity types (Step 2)

Each weak entity type takes its owner's primary key as a foreign key. Its primary key is the owner's key plus the partial key:

Weak entity	Owner	Primary key
CATEGORY	GAME	(game_id, category_no)
LEVEL	GAME	(game_id, level_no)
THREAD	FORUM	(forum_id, thread_code)

Weak entity	Owner	Primary key
REPLY	THREAD	(forum_id, thread_code, reply_no)

1.4 Composite and derived attributes

Composite attributes became their component columns:

- GAME.ruleset → show_milliseconds, require_video, emulators_allowed, default_time_type
- USER.location → country, subregion
- run_time, inside RUN.times → hours, minutes, seconds, milliseconds (in RUN_TIME, see §1.5)

Derived attributes, not stored. The site computes each of these from stored data:

Entity	Derived attributes
GAME	total_runs, follower_count
USER	total_runs, world_records, account_age_days
RUN	leaderboard_rank, is_world_record, is_obsolete
THREAD	thread_path, reply_count

1.5 Multi-valued attributes (Step 6)

Attribute	Option	Result	Reason
GAME.run_time_types	(a), max 3	run_time_type_1, run_time_type_2, run_time_type_3 in GAME (2 and 3 nullable)	The time_type domain has exactly 3 values, each used at most once (Team 67 §2.4)
GAME.genres	(b)	GAME_GENRE(game_id, genre)	No maximum given
GAME.developers	(b)	GAME_DEVELOPER(game_id, developer)	No maximum given
GAME.links	(b)	GAME_LINK(game_id, link)	No maximum given
USER.social_links	(b)	USER_SOCIAL_LINK(user_id, link)	No maximum given
RUN.video_links	(b)	RUN_VIDEO_LINK(run_id, link)	No maximum given ("one or more links")
RUN.times	(b)	RUN_TIME(run_id, time_type, hours, minutes, seconds, milliseconds)	Has sub-attributes (time_type, run_time)
MODERATOR_ROLE.permissions	(b)	ROLE_PERMISSION(role_id, permission)	No maximum given
NEWS_POST.topics	(b)	NEWS_POST_TOPIC(news_id, topic)	No maximum given

Notes on these mappings:

- **RUN_TIME key.** The primary key is (run_id, time_type), not all six columns. A run has at most one time per time type (Team 67 §2.2; Team 63 M4), so time_type alone identifies an entry within a run.
- **Time-type slots in GAME.** The order of run_time_type_1/2/3 has no meaning; the slots hold an unordered set. The game's default is stored separately in default_time_type.

[ALT-(b)] *Use this row and note instead if the team chooses option (b).*

- Table row: *GAME.run_time_types | (b) | GAME_RUN_TIME_TYPE(game_id, run_time_type) | Kept as a set so it can be compared with RUN_TIME.time_type without ordered slots.*
- Note: *The rule that default_time_type must be one of the game's types is then a cross-relation rule, enforced by a trigger.*

1.6 Binary relationship types (Steps 3, 4, 5)

The FK column for each 1:N relationship goes on the N side; "(NULL)" marks partial participation of that side.

#	Relationship	Ratio	Mapping
1	PART_OF	M:N	PART_OF(game_id, series_id)
2	BASE_GAME_OF (recursive)	1:N	GAME.base_game_id → GAME (NULL)
3	HAS_CATEGORY (identifying)	1:N	Part of CATEGORY's key (§1.3)
4	HAS_LEVEL (identifying)	1:N	Part of LEVEL's key (§1.3)
5	AVAILABLE_ON	M:N	AVAILABLE_ON(game_id, platform_id)
6	RELEASED_IN	M:N	RELEASED_IN(game_id, region_id)
9	FOLLOWS (recursive)	M:N	FOLLOWS(follower_id, followed_id), both → USER
10	FOLLOWS_GAME	M:N	FOLLOWS_GAME(user_id, game_id, last_visit)
11	FEATURES	M:N	FEATURES(user_id, run_id, featured_order)
12	RUN_IN	1:N	RUN.(game_id, category_no) → CATEGORY
13	RUN_FOR_LEVEL	1:N	LEVEL_RUN.(game_id, level_no) → LEVEL
14	SUBMITS	1:N	RUN.submitter_id → USER
15	PLAYED_BY	M:N	PLAYED_BY(run_id, user_id, player_order)
16	EXAMINES	1:N	RUN.examiner_id → USER (NULL)
17	PLAYED_ON	1:N	RUN.platform_id → PLATFORM
18	PLAYED_IN	1:N	RUN.region_id → REGION (NULL)
19	HAS_FORUM	1:1	GAME.forum_id → FORUM, NOT NULL and UNIQUE
20	FORUM_HAS_THREAD (identifying)	1:N	Part of THREAD's key (§1.3)
21	STARTED_BY	1:N	THREAD.starter_id → USER
22	WRITTEN_BY	1:N	NEWS_POST.author_id → USER
23	APPROVED_BY	1:N	NEWS_POST.approver_id → USER (NULL)
24	NEWS_ABOUT	1:N	NEWS_POST.game_id → GAME (NULL)
25	STREAMS_GAME	1:N	STREAM.game_id → GAME
26	HOLDS_MEMBERSHIP	M:N	HOLDS_MEMBERSHIP(user_id, plan_id, start_date, end_date)
27	RAISES	1:N	TICKET.raised_by → USER

#	Relationship	Ratio	Mapping
28	HANDLED_BY	1:N	TICKET.handler_id → USER (NULL)
29	TICKET_ABOUT	1:N	TICKET.game_id → GAME (NULL)
30	HAS_REPLY (identifying)	1:N	Part of REPLY's key (§1.3)
31	POSTED_BY	1:N	REPLY.author_id → USER
F1	REPORTS_RUN (Team 63's fix)	1:N	TICKET.run_id → RUN (NULL)

HAS_FORUM uses the foreign-key approach. The FK goes in GAME because GAME participates totally, which makes it NOT NULL. UNIQUE enforces the 1:1 ratio. Site-wide boards are FORUM rows that no GAME references.

NOTE: Relationships 7&8 (ternary) are listed below (section 1.7).

1.7 Ternary relationship types (Step 7)

MODERATES_GAME became MODERATES_GAME(user_id, game_id, role_id, since), with FKs to USER, GAME and MODERATOR_ROLE.

- The ratio is M:N:1, with the 1 on MODERATOR_ROLE (Team 63 M1).
- So the primary key leaves out role_id and is (user_id, game_id). The textbook rule: when an entity type has max 1, its FK is not part of the key.

MODERATES_SERIES is mapped the same way, with primary key (user_id, series_id).

1.8 Specialization (Step 8)

LEVEL_RUN is a partial specialization of RUN with one subclass, and has no attributes of its own.

- We used **Option 8A** (multiple relations: superclass and subclasses).
- The result is LEVEL_RUN(run_id, game_id, level_no):
- run_id is its primary key and an FK to RUN.
- (game_id, level_no) is the FK of RUN_FOR_LEVEL to LEVEL.
- 8C/8D would add a nullable level column and type field to every run. 8B is not allowed for a partial specialization.

1.9 Union types (Step 9)

The diagram has no union types (categories). Its "`U`" symbol marks the LEVEL_RUN subclass.

1.10 Constraints added while mapping

1. **TICKET:** `CHECK (game_id IS NULL OR run_id IS NULL)`. A run-report ticket gets its game through RUN. This fills a gap in the diagram; see §6.
2. **GAME:** default_time_type must equal one of run_time_type_1/2/3 (Team 67, I6), and the non-NULL slots must be distinct.

- A plain `default_time_type IN (run_time_type_1, run_time_type_2, run_time_type_3)` is unsafe. When it fails to match, a NULL slot makes the result UNKNOWN, and a CHECK accepts UNKNOWN.

- So compare against `COALESCE(run_time_type_2, run_time_type_1)` and `COALESCE(run_time_type_3, run_time_type_1)` instead.

3. *(Optional)* **TICKET:** `CHECK (run_id IS NULL OR ticket_type = 'run report')`.
4. *(Optional)* **LEVEL_RUN:** declare UNIQUE (run_id, game_id) on RUN, and add FK (run_id, game_id) → RUN on LEVEL_RUN. Together with FK (game_id, level_no) → LEVEL, this enforces Team 67 §2.3 #13: the level belongs to the same game as the run's category. It also keeps LEVEL_RUN.game_id consistent with RUN.game_id.

Notation: Elmasri & Navathe relational schema notation. Primary keys are underlined, and every foreign key has an arrow to the primary key it references.

[State any deviation in one line, e.g. composite-FK arrows drawn from the group of columns.]

Stage A snapshot

Please see the first attachment at the bottom of the file.
Link: FULL URL

2. Stage B: First Normal Form

No conversion needed. Stage A already produces atomic columns, and no column holds a composite value or a list:

- Composite attributes became separate columns (GAME.ruleset, USER.location, the run_time inside RUN.times).
- Multi-valued attributes were mapped under Stage A, item 4. Eight became separate relations, and run_time_types became a fixed set of three columns under option (a), which the spec accepts as 1NF.

[ALT-(b)] *"…and all nine multi-valued attributes became separate relations."*

3. Stage C: Second Normal Form

No conversion needed. No relation has a non-prime attribute that depends on only part of a candidate key:

- **Single-attribute candidate keys only, so a partial dependency is impossible:** FORUM, NEWS_POST, MEMBERSHIP, USER, TICKET, RUN, LEVEL_RUN, REGION, STREAM, SERIES, PLATFORM, GAME, MODERATOR_ROLE.
- **All-key, so there are no non-prime attributes:** NEWS_POST_TOPIC, USER_SOCIAL_LINK, RUN_VIDEO_LINK, GAME_GENRE, GAME_DEVELOPER, GAME_LINK, ROLE_PERMISSION, PART_OF, AVAILABLE_ON, RELEASED_IN, FOLLOWS.
- **Composite keys with non-key attributes:** CATEGORY, LEVEL, THREAD, REPLY, RUN_TIME, MODERATES_GAME, MODERATES_SERIES, FOLLOWS_GAME, FEATURES, PLAYED_BY, HOLDS_MEMBERSHIP. Each non-key attribute needs the whole key; §5 shows this relation by relation.
- THREAD and REPLY depend on thread_code being unique only within its forum (Team 67 §2.4), so thread_code does not determine forum_id.

4. Stage D: Third Normal Form

4.1 What changed and why

Two relations change, RUN and TICKET. In each, the stored `status` is functionally determined by other non-key columns.

RUN: examiner_id, reject_reason → status.

status	examiner_id	reject_reason	Source
new	NULL	NULL	Examiner set only once a run is decided (Team 67 §2.3 #16)
verified	set	NULL	U1 fills only the examiner and date
rejected	set	set	U2 and U3 fill the examiner and reason; a rejected run must have a reason (§2.4 rule 4)

The three rows have pairwise different NULL patterns, so two runs that agree on (examiner_id, reject_reason) have the same status. This holds even with NULL compared as an ordinary value, since every pending run is (NULL, NULL) and new.

TICKET: handler_id, closed_at → status.

status	handler_id	closed_at	Source
open	NULL	NULL	I4: created with no handler
in progress	set	NULL	U5: taking a ticket records the handler
closed	set	set	U5: closing sets closed_at; §2.4 rule 4: a handler from in progress onwards

Why both violate 3NF.

- **The left side is not a superkey.** Every pending run has (NULL, NULL), and so does every open ticket. Many rows share the same left-side values.
- **status is not prime.** The only candidate keys are run_id and ticket_id.
- **The resulting anomaly:** a run can be given an examiner while its status still says `new`, and the two copies of the fact then disagree.

Fix. We removed `status` from RUN and TICKET. Each is recovered exactly by a view:

```
-- RUN_WITH_STATUS
CASE WHEN examiner_id IS NULL THEN 'new'
WHEN reject_reason IS NULL THEN 'verified'
ELSE 'rejected' END
-- TICKET_WITH_STATUS
CASE WHEN closed_at IS NOT NULL THEN 'closed'
WHEN handler_id IS NOT NULL THEN 'in progress'
ELSE 'open' END
```

We did not decompose, because the textbook decomposition does not work here. It would create RUN_STATUS(examiner_id, reject_reason, status). That relation has no usable primary key, since both columns are NULL for every pending run. It would also record no fact beyond the three-case rule above. Dropping the column is lossless: each status matches exactly one NULL pattern, so the view returns what the column held.

CHECK constraints added to keep the NULL patterns valid:

- RUN:
- `(examiner_id IS NULL) = (decided_on IS NULL)`
- `reject_reason IS NULL OR examiner_id IS NOT NULL`
- TICKET:
- `closed_at IS NULL OR handler_id IS NOT NULL`
- `(closed_at IS NULL) = (resolution IS NULL)`

What stays the same:

- **Requirements that refer to status** use the view or the NULL test. Pending runs (R2) are `examiner_id IS NULL`. "Verify only while new" (§1.3, U1) becomes `UPDATE … WHERE examiner_id IS NULL`.
- **NEWS_POST.status is kept.** Pending and rejected posts both have approver_id and approved_at NULL (U4), so no other column determines status.

4.2 Result

After this change, no relation has a nontrivial FD X → A where X is not a superkey and A is non-prime (see §5). The model is in 3NF.

Stage D snapshot

Please see the second attachment at the bottom of the file.
Link: FULL URL

FDs are given relation by relation, with all candidate keys (CK) of each. "Checked" lists the dependencies we tested and rejected, each with the reason from the requirements. A CK determines every attribute of its relation, so "K → all" is not repeated for each one.

5.1 Relations changed in Stage D

Relation	CKs	FDs used / checked
RUN	{run_id}	run_id → all. **examiner_id, reject_reason → status** (violation, §4.1; equivalently decided_on, reject_reason → status). After the change: no FD other than run_id → all.
TICKET	{ticket_id}	ticket_id → all. **handler_id, closed_at → status** (violation, §4.1). Checked: run_id → game_id does not hold, because CHECK (game_id IS NULL OR run_id IS NULL) means a run's ticket stores no game. After the change: no FD other than ticket_id → all.

5.2 Relations with one or more single-attribute keys

Relation	CKs	FDs checked
FORUM	{forum_id}, {forum_name}	forum_id → forum_name; forum_name → forum_id. Both left sides are keys.
NEWS_POST	{news_id}	approver_id, approved_at → status does not hold: pending and rejected posts are both (NULL, NULL) (U4).
MEMBERSHIP	{plan_id}	plan_name → monthly_price is not given by the requirements. If plan names are unique, plan_name is a second CK and the FD's left side is a key.
USER	{user_id}, {username}, {email}	subregion → country does not hold: subregion is a region name meaningful only together with country (e.g. Punjab occurs in two countries). No FD among the other non-key attributes.
LEVEL_RUN	{run_id}	run_id → game_id, level_no. No other FD.
REGION	{region_id}	Binary relation, so in 3NF. If region names are unique, region_name is a second CK.
STREAM	{stream_id}	channel_name, service → title, game_id not assumed: a channel can have several streams listed, each with its own title and game. If (channel_name, service) is unique, it is a second CK.
SERIES	{series_id}	Binary relation.
PLATFORM	{platform_id}	platform_name → release_year: same reasoning as MEMBERSHIP.
GAME	{game_id}, {abbreviation}, {forum_id}	run_time_type_1 → default_time_type does not hold. Slot order has no meaning, so two games can both have run_time_type_1 = real time with different defaults. No FD among the other non-key attributes. forum_id is a CK because HAS_FORUM is 1:1 with GAME total (NOT NULL, UNIQUE).
MODERATOR_ROLE	{role_id}, {role_name}	role_id → role_name; role_name → role_id.

5.3 Composite keys with non-key attributes

Relation	CK	FDs checked
CATEGORY	{game_id, category_no}	game_id →/name, type, rules, is_misc (a game has many categories). category_no →/same (numbered within each game). No FD among name, type, rules, is_misc.
LEVEL	{game_id, level_no}	game_id →/name, rules; level_no →/name, rules (same reasons).
THREAD	{forum_id, thread_code}	thread_code →/forum_id (unique only within its forum, Team 67 §2.4). Neither key attribute alone determines title, created_at, is_locked, starter_id.
REPLY	{forum_id, thread_code, reply_no}	forum_id, thread_code →/body, posted_at, edited_at, author_id (a thread has many replies). reply_no →/same (numbered within each thread). thread_code →/forum_id.
RUN_TIME	{run_id, time_type}	run_id →/hours, minutes, seconds, milliseconds (up to one time per type). time_type →/same.
MODERATES_GAME	{user_id, game_id}	user_id, game_id → role_id, since (M:N:1). user_id →/role_id (different roles on different games); game_id →/role_id.
MODERATES_SERIES	{user_id, series_id}	Same as MODERATES_GAME, with series in place of game.
FOLLOWS_GAME	{user_id, game_id}	user_id →/last_visit; game_id →/last_visit.
FEATURES	{user_id, run_id}	user_id →/featured_order; run_id →/featured_order (a run can be featured by several of its players).
PLAYED_BY	{run_id, user_id}	run_id →/player_order; user_id →/player_order.
HOLDS_MEMBERSHIP	{user_id, plan_id}	One link per user and plan (Team 67 §2.3 #26). user_id →/start_date, end_date; plan_id →/same.

5.4 All-key relations

Relations: NEWS_POST_TOPIC, USER_SOCIAL_LINK, RUN_VIDEO_LINK, GAME_GENRE, GAME_DEVELOPER, GAME_LINK, ROLE_PERMISSION, PART_OF, AVAILABLE_ON, RELEASED_IN, FOLLOWS.

CK: all attributes, in each relation.

Why they are in 3NF. Each of these relations has exactly two attributes, so the only possible nontrivial FDs are A → B or B → A. If either held, its left side would be a superkey (it would determine every attribute), so the FD would not violate 3NF. If neither holds, there are no nontrivial FDs, and the candidate key is both attributes. Either way, the relation is in 3NF.

6. Changes to the assigned EER diagram

1. **TICKET: added `CHECK (game_id IS NULL OR run_id IS NULL).`** TICKET_ABOUT and REPORTS_RUN (F1) both allow a ticket to name a game and a run at once. A run-report ticket's game is taken from the run, so storing it again would be redundant and could contradict the run's game.
2. *(Only if kept)* **TICKET: added `CHECK (run_id IS NULL OR ticket_type = 'run report').`** Only run reports name a run, which matches the reason REPORTS_RUN was added (Team 63, C2/F1).

The removal of RUN.status and TICKET.status is a normalization step (Stage D), not a change to the diagram: status stays available through the views, so no information from the diagram is lost.

7. GenAI usage statement

We used Claude (Anthropic) to check the normal form of our mapped schema against Team 67's requirements, to find the functional dependencies discussed in §4 and §5, and to draft the outline of this report. The team made every mapping and design decision, checked each dependency against the source documents, drew all diagrams, and reviewed the final text.



